Компьютерные науки и анализ данных

Yandex

Faculty of Computer science
Higher School of Economics

Программирование на Python

лектор: Денис Беляков

План

- Методы класса и статические методы
- Магические методы
- Приватные/защищенные атрибуты
- property
- Класс как пространство имён
- Наследование в Python
- Полиморфизм в Python

Методы класса и статические методы

- Методы класса – работают с классом, а не экземпляром класса. Нулевым аргументом метода будет cls – текущий класс. Методы класса задаются декоратором @classmethod
- Статические методы – не обращаются к полям класса и экземпляра. Задаются декоратором @staticmethod, не принимают никакого нулевого аргумента

Ещё о magic методах

- __eq__, __lt__, __gt__, __ne__, __le__, __ge__ – сравнение
- *Можем задать __eq__ и один из операторов <,<=,>, >= и поставив перед классом декоратор @functools.total_ordering, он определит всё остальное*
- __len__
- __getitem__ и __setitem__, __delitem__ – работа с [] в словарях
- __hash__
- __call__
- __bool__
- __iter__, __next__
- __dict__

Приватные/защищенные атрибуты

- В питоне принято начинать название атрибута с _ – уровень protected на согласовании на уровне разработчика (его не надо использовать вне класса, только внутри). Но можем спокойно обратиться извне класса
- Приватные атрибуты – начинаются с __. К ним уже нельзя обратиться вне класса (AttributeError). Сокрытие обработки защищенных атрибутов. При этом предоставляем публичный метод (часто обусловлено требованием консистентности данных!)
- Приватные методы также должны начинаться с __
- Но функция dir может спалить эти защищенные атрибуты… 🤷 Таким образом в питоне нельзя сделать честное приватное поле. Но вроде как есть модуль accessify, в котором есть декораторы protected и private

property (свойство)

- Позволяет использовать методы в качестве свойств объектов — порождает дескриптор, позволяющий создавать «вычисляемые» свойства (тип property).
- Можно использовать @property (по умолчанию, только для чтения)
- Объект свойства также предоставляет методы getter, setter, deleter, которые можно использовать в качестве декораторов для указания функций реализующих получение, установку и удаление свойства соответственно.
- Дескриптор (подробная статья):
  - Атрибут объекта, доступ к которому подчиняется протоколу дескриптора.
- Для реализации дескриптора классу требуется определить один или несколько методов, входящих в протокол дескриптора:
  - __get__(), __set__(), __delete__(), __set_name__()

Класс как пространство имен

- Как же можно внутри класса обратиться к атрибутам класса??

1. Через self. Так как этого метода нет в экземпляре, он будет искаться в классе
2. Через имя класса (имена доступны)
3. Декоратор property и self
4. создать classmethod – обращение через cls
5. staticmethod – через имя класса

Новый взгляд и обобщение вышесказанного

- Класс — это пространство имён
- Класс — это изготовитель (чтобы не сказать «конструктор») объекта — экземпляра класса, т. е. такой callable. У класса (объекта) есть поля(атрибуты). Callable-поля называются методы, не-callable – поля.
- Все поля класса видны в объекте
- До тех пор, пока их не загородят уникальные поля объекта с теми же именами (аналог: глобальные и локальные переменные)
- При этом функции класса превращаются в методы объекта
- Метод — это такая функция внутри пространства имён объекта, при вызове которой в начало кортежа параметров ставится сам объект
- Специальный метод __init__() нужен для создания полей объекта и вообще логики его создания. Это не конструктор!

Полиморфизм в Python

- Пример: помещаем объекты разных классов в список. Причем классы имеют методы с одним и тем же названием, но разной имплементацией
- Если вызывать этот метод от объектов, итерируясь по списку, то всё отработает корректно!
- С виду мы обращаемся к одному и тому же методу, но питон будет сам понимать, у объекта какого класса он будет вызывать метод! Это полиморфное поведение
- Аналогично с магическими методами

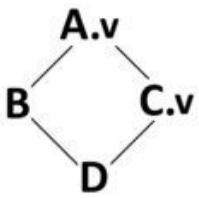
Наследование

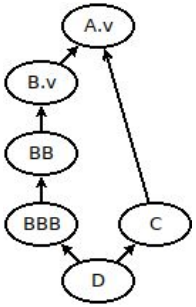
- Наследование классов в Python происходит путем перечисления классов-родителей в скобках после объявления имени класса
- `isinstance(ex, Class)` – является ли ex экземпляром Class или его потомков
- `type(ex) == Class` – точное сравнение: является ли ex экземпляром Class
- `issubclass(Class, BaseClass)` – является ли класс потомком заданного
- В классах-наследниках можно определять новые методы или переопределять методы родительского класса.
- Все классы в Python унаследованы от класса object
- Иногда бывает проблематично унаследоваться от объектов базовых типов. Но на помощь (неожиданно) может прийти модуль collections

Наследование

- Видимость для наследника:
  - Поля объекта
  - Поля класса
  - Поля базового класса
- Идиома для вызова конструктора текущего класса – `type(self)`
- `super()` возвращает пространство имён, содержащее поля базового класса
- при создании super()-объекта ***не создаётся*** экземпляр базового класса (например, не вызывается __new__() и т. п.)

Проблема ромбовидного наследования:





Обход в глубину (слева) добирается до A.v раньше, чем до C.v
Обход в ширину (справа) добирается до A.v раньше, чем до B.v

Множественное наследование и mro
`(__mro__())` – линеаризация

Higher School of Economics
Faculty of Computer science

Faculty of Computer Science
Higher School of Economics

sgorshkov@hse.ru
tg: @Seragor98